



University of Stuttgart
Germany

AI Planning

Ebaa Alnazer

ebaa.alnazer@iaas.uni-stuttgart.de

Room: U38.03

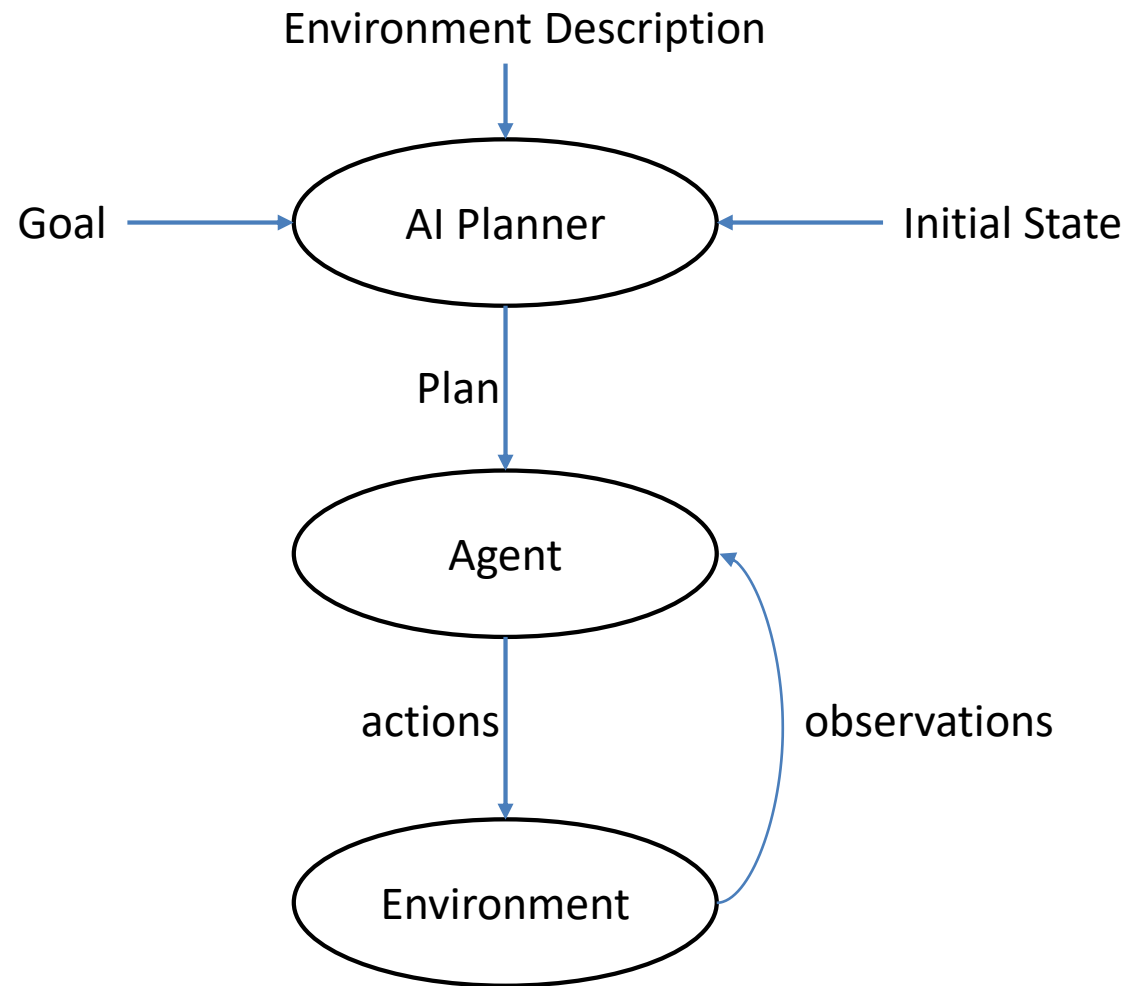
2026

Summer

What is AI Planning?

A Sub-area of artificial intelligence that studies the decision making process computationally in order to empower **agents** to pursue their **goals**

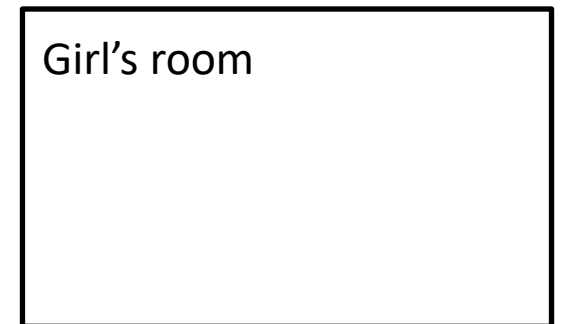
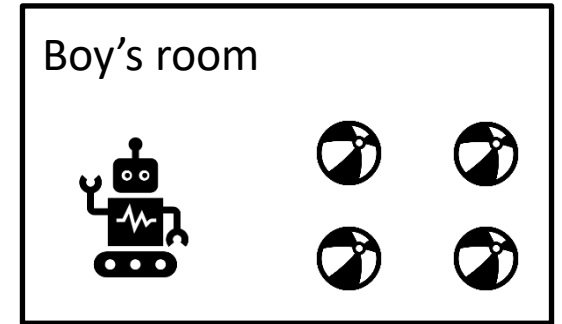
What is AI Planning?



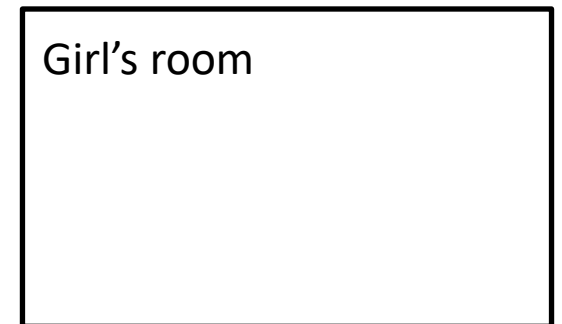
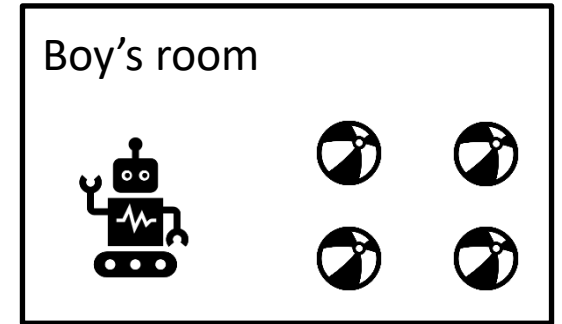
Example

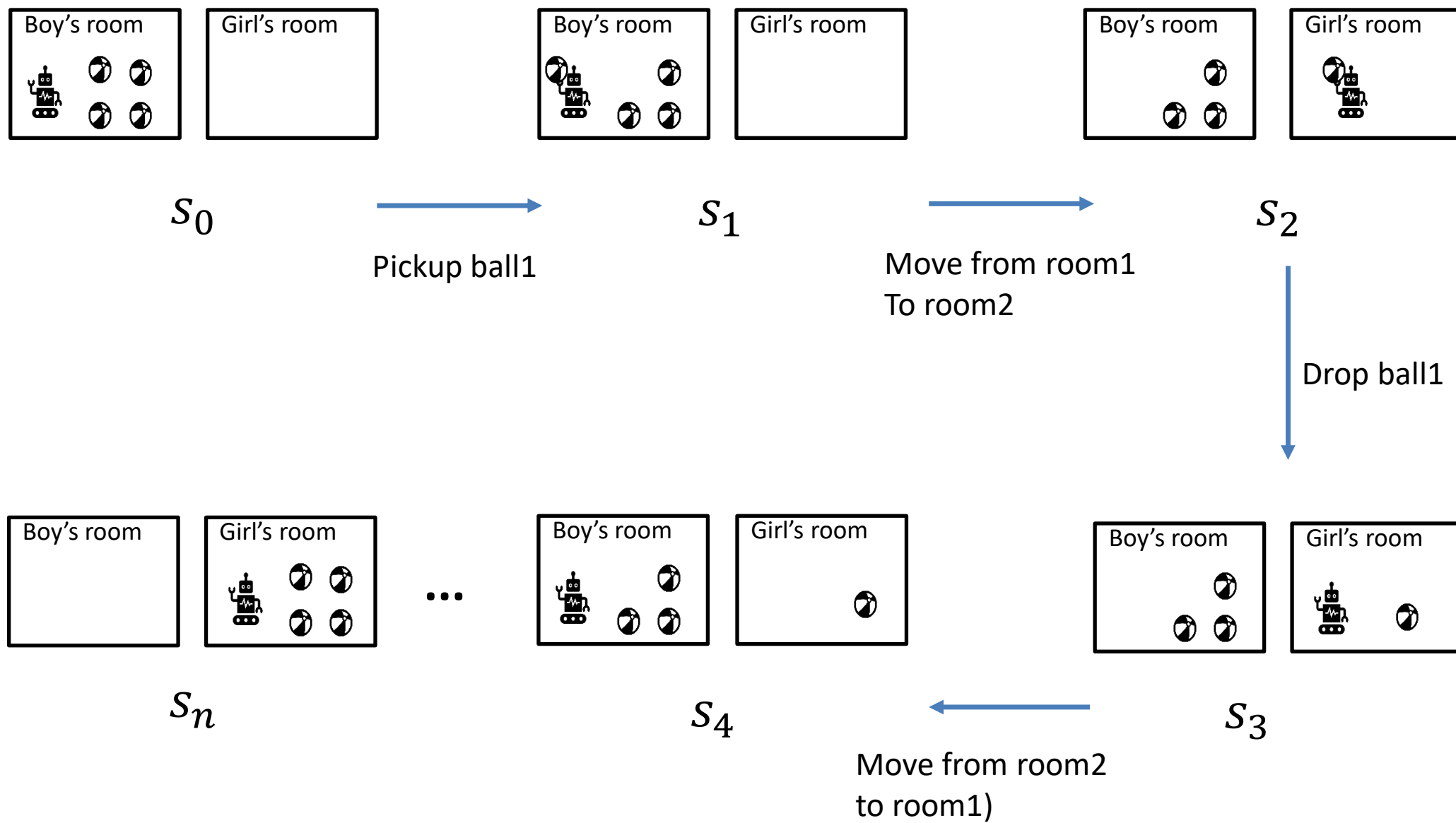
TARS:THE GRIPPER

- **Environment**
 - The rooms, the robot, the possible actions, possible states...
- **Agent**
 - The robot
- **Initial state**
 - The robot and the balls in the boy's room
- **Actions**
 - Pickup a ball
 - Drop a ball
 - Move to the other room



- Goal
 - The robot and the balls in the girl's room
- Plan
 - Pickup ball I – move to the girl's room – drop ball I - move to the boy's room ...

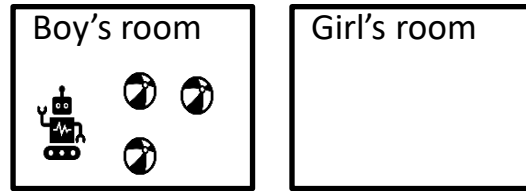




Closed-World Assumption

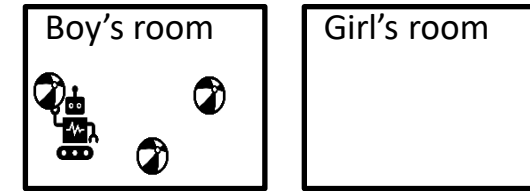
If an object statement/atom is not explicitly specified in a state, it means that it does not hold in that state

Closed-World Assumption



S_0

Pickup ball1



S_1

- Ball1 is in boy's room
- ~~Not (Ball1 is in girl's room)~~
- Ball2 is in boy's room
- ~~Not (Ball2 is in girl's room)~~
- Ball3 is in boy's room
- ~~Not (Ball3 is in girl's room)~~
- TARS' arm is empty
- TARS is in boy's room
- ~~Not (TARS is in girl's room)~~

- ~~Not (Ball1 is in boy's room)~~
- Ball2 is in boy's room
- Ball3 is in boy's room
- ~~Not (TARS' arm is empty)~~

Classical Planning (Recap)

- Domain-independent planning
- Assumptions:
 - Environment has a finite set of states,
 - Fully observable,
 - Static,
 - Deterministic,
 - Goals are either satisfied or not,
 - Plans are ordered sequence of actions, and
 - No representation of time

Classical Planning (Recap)

- Classical planning problem

$$P = \langle F, O, I, G \rangle$$

- F : a set of atoms:

ball 1 is in room 1, TARS is in room 2,...

- O : a set of operators, such that
an operator $o \in O$ is a triple $\langle \text{pre}(o), \text{add}(o), \text{del}(o) \rangle$

pickup ball b operator:

pre: b in room x, TARS in room x

add: TARS' arm is not empty

del: b in room x, TARS' arm is empty

(not necessary)



Classical Planning (Recap)

- Classical planning problem

$$P = \langle F, O, I, G \rangle$$

- $I \subseteq F$: initial state

ball1 is in room1, ball2 is in room1, TARS is in room1,...

- $G \subseteq F$: Goal state.

ball1 is in room2, ball2 is in room2, TARS is in room2,...

Classical Planning (Recap)

- Plan

$$\pi = o_1, o_2, \dots, o_n$$

- A sequence of operators.
- o_i is applicable in s_i ($pre(o_i) \subseteq s_i$), and
- $G \subseteq s_1[\pi]$

pickup ball1, move to room2, drop ball1, move to room1, ..., drop ball4

Applicability of Operators

- *Pickup a ball **b*** operator
- The pickup operator's preconditions:
 - **b** is in room **x**
 - TARS is in room **x**
 - TARS' arm is empty
- *Pickup **ball1***
- The current state is:
 - **ball1** is **room1**
 - TARS is in **room1**
 - TARS' arm is empty
 - ...
- b/ball1
- x/room1

Solving Planning Problems

- How can the agent determine what to do in order to achieve its goals?
 - An **algorithm /search strategy/ search technique** to look for the solution in the **search space**
- Different search strategies work in different search spaces
 - Here we will study the **state-space planning**

State-Space Planning

- Searching in a subset of state space
- The planning task is represented as a directed graph:
 - Nodes represent world states
 - Edges represent actions
 - A plan: is a path through the state space
- We study two different strategies:
 - Forward search
 - Backward search

Forward Search

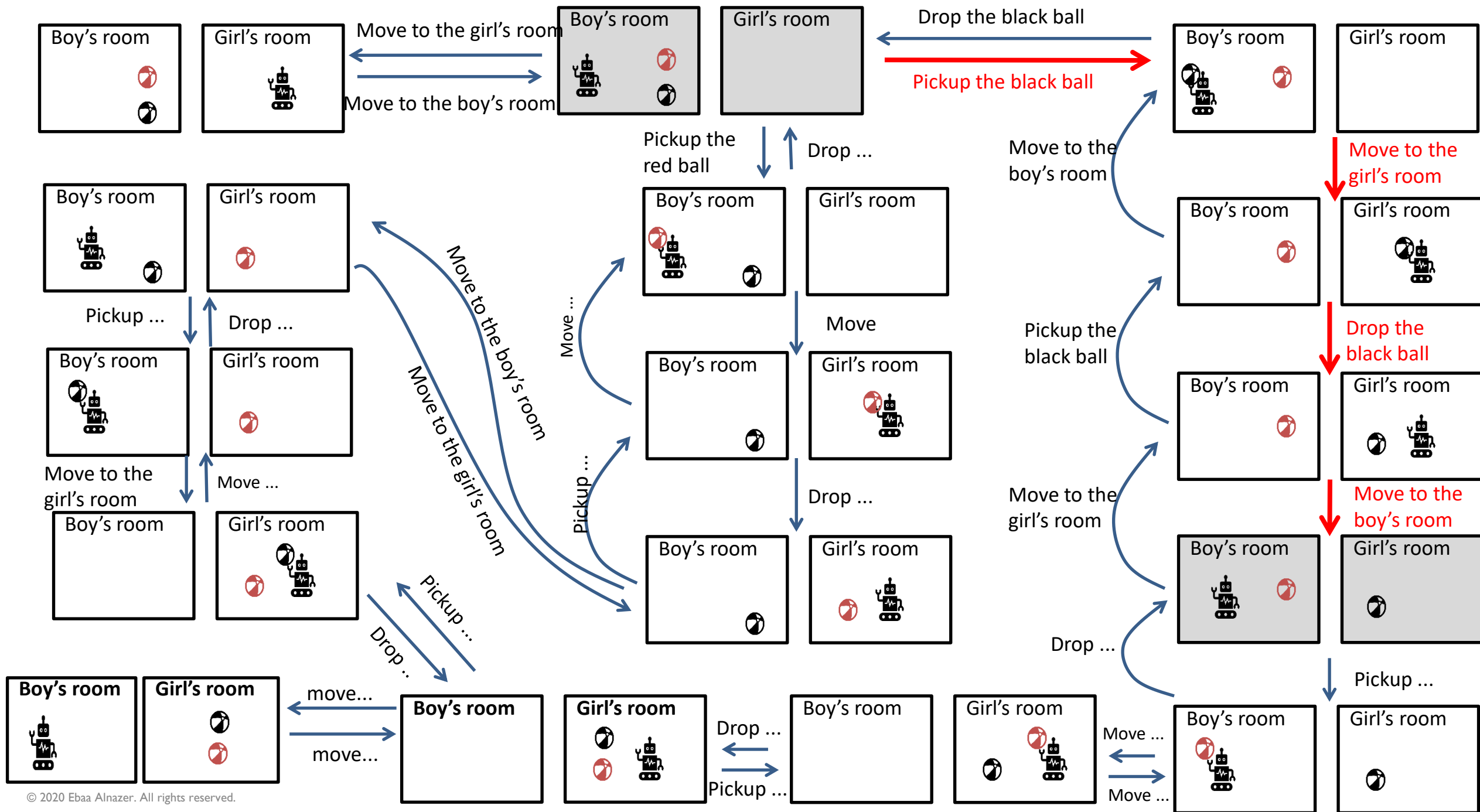
- Starts from the initial state and search the space for a state that satisfies the goal state.
- At each state, the algorithm searches for applicable operators
 - The choice of an operator is done **non-deterministically**
 - The next state is the state resulting from the chosen operator application in the current state.
- The algorithm terminates if:
 - It reaches a state that satisfies the goal
 - No solution is found

Example

TARS:THE GRIPPER

TARS: the gripper

- We have a boy's room, a girl's room, TARS (the smart home robot), a red ball and a black one
- Initially, the balls and TARS are in the boy's room
- The goal is to have the black ball in the girl's room and TARS and the red ball in the boy's room
- TARS can grab a ball using one of its two hands



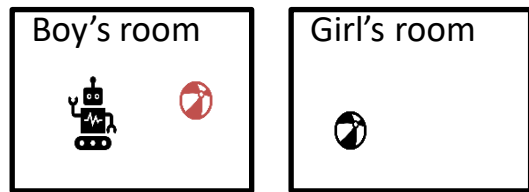
Forward Search

- The algorithm is **sound** and **complete**.
- Factors affecting the search space:
 - Number of rooms
 - Number of balls
 - Number of robots
 - ...
- Inefficient:
 - Explores irrelevant actions/ has a large *branching factor*
 - State spaces often are large even for small problems
- Needs a good heuristic and/or pruning strategies

Backward Search

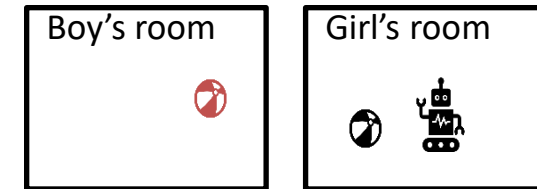
- Starts from the goal state and searches backwards by applying actions that are **relevant** to each state until reaching the initial state.
- An action a is relevant to state s if and only if:
 - a makes at least one literal of s true
 - The effects of a do not contradict any literal in s
- The group of states obtained by applying actions backwards from state g is called (Regression Set) g' ,
$$g' = (g - add(a)) \cup pre(a)$$

Backward Search



- TARS is in the boy's room
- The red ball is in the boy's room
- The black ball is in the girl's room

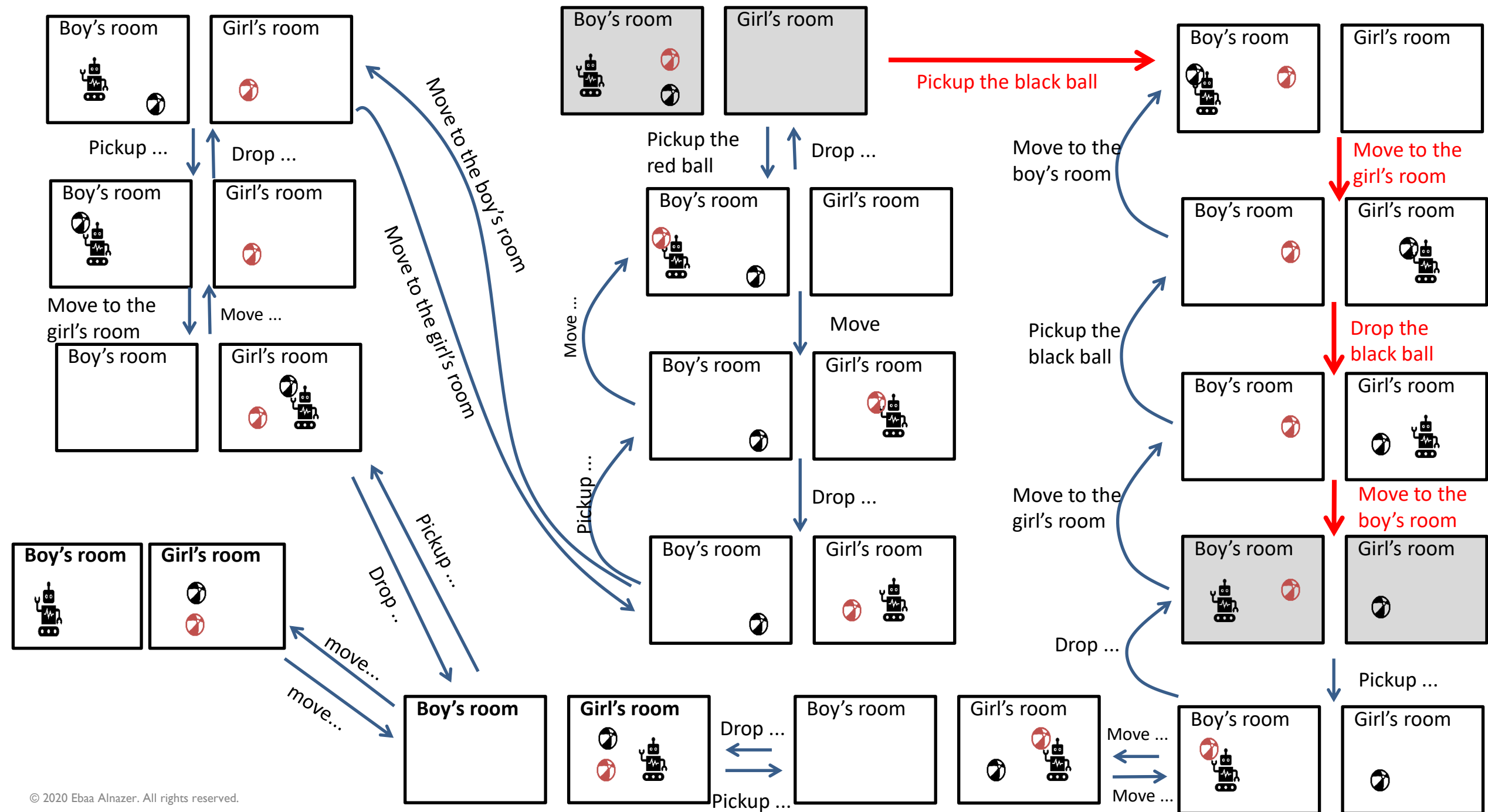
Move from the girl's room to the boy's room



- Move TARS from x to y
 - Pre: TARS is in x
 - Add: TARS is in y
 - Del: TARS is in x
- The substitution:
 - y /boy's room
 - x /girl's room

- TARS is in the girl's room
- The red ball is in the boy's room
- The black ball is in the girl's room

- Delete the add list: TARS is in boy's room
- Add the precondition: TARS is in girl's room

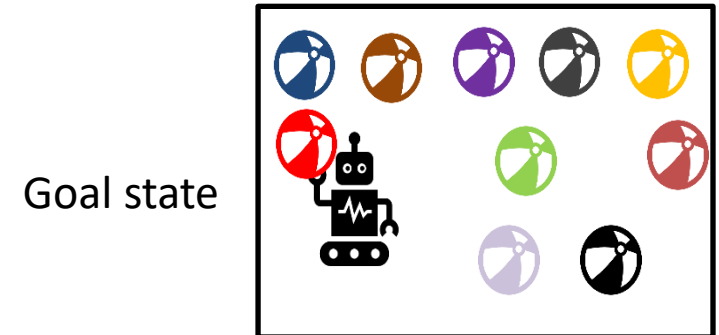
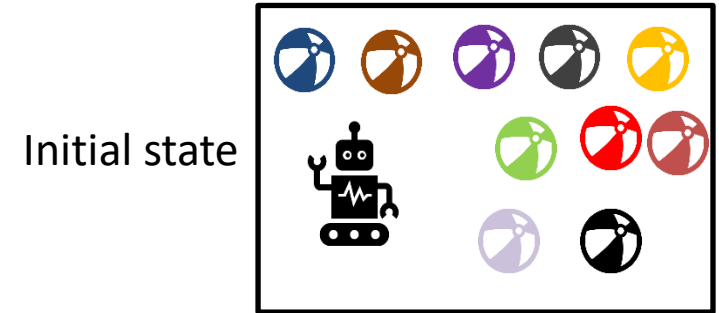


Backward Search

- The algorithm is **sound** and **complete**.
- Factors affecting the search space:
 - Number of rooms
 - Number of balls
 - Number of robots
 - ...
- In general, backward search has a smaller search space
 - Considers only relevant actions
- The branching factor is usually smaller than forward search
- The usage of state sets rather than individual states makes it harder to come up with good heuristics

Which strategy to use?

- Depends on the type of the problem
 - In this example, backward search works better.
 - In chess playing, forward search is better.
- The branching factor → backward
- Explaining the result → backward
- ...



Exercise

A house has three rooms, R1, R2 and R3, and two boxes full of LEGO bricks, L1 and L2. R1 is adjacent to R3 and R2 is adjacent to R3. TARS, a domestic robot, can push a LEGO box x from room y to room z , given the box and TARS are at y and y is different from z . TARS can move from x to y if it is at x , x is different from y and x is adjacent to y . In the initial state, L1 is at R1 and L2 is at R3, and the robot is at R2. The goal is to have the LEGO boxes at R2.

1. List all negated atoms that are true in the initial state.
2. Describe the state resulting from moving TARS to R3 in the initial state.
3. Show the first expansion in the search space using backward search.
4. Specify a plan that will satisfy the goal.

References

- Aiello, M. and Georgievski, I. Artificial Intelligence Planning in Planning for Pervasive Systems, Lecture Notes, 2019.
- Norvig, P. Russel, and S. Artificial Intelligence. *A modern approach*. Prentice Hall, 2002.
- Rintanen, J. and Ragni, M. Foundations of Artificial Intelligence Course Slides, Freiburg University, 2005.

